

NLP Project Progress Update

TempoRAG-KG

Temporal Knowledge Graph-Augmented Retrieval
for Multi-Hop Question Answering

Supanut Kompayak · Aphisit Jaemyaem · Dechathon Niamsa-ard · Kaung Hein Htet

Asian Institute of Technology

1 / 7

Good Morning

From Wikipedia QA to SEC 10-K filings

Dataset pivot: anchoring problem → year-stamped corpus

	Proposal round	Progress round
Dataset	HotpotQA + MuSiQue (Wikipedia multi-hop)	25 × 10-K filings (5 tech mega-caps × 5 fiscal years)
Core limitation we hit	Facts lack explicit year anchors; Wikipedia spans decades with no canonical "as of" date	Every 10-K is filed for a specific fiscal year — year is metadata, not inference
Why it matters for us	Can't evaluate temporal filtering when ground-truth time itself is ambiguous	Temporal correctness is testable — filter right year, get right fact

Before we dive into the progress report, let me quickly walk you through what changed once we started building.

In the proposal, our plan was to evaluate temporal retrieval on HotpotQA and MuSiQue — two standard multi-hop QA benchmarks built on Wikipedia.

But once we set up the evaluation, we hit a problem: the questions in these datasets don't really carry timestamps. A question like *"Who is Apple's CEO?"* doesn't specify *when* — and Wikipedia itself has no canonical "as-of" date, because the pages keep getting rewritten.

So we discussed this with our TA, Gun. He pointed out that these aren't really the right datasets for evaluating temporal question answering — the ground truth's time is itself ambiguous — and he recommended we move to a corpus with a cleaner temporal signal: SEC 10-K filings, which are public financial disclosures from the tech sector.

That's what you see on the right. We're now working with 25 filings — five tech mega-caps over five fiscal years. Every 10-K is filed for a specific fiscal year, so the year lives in the filing metadata, not in the text. That means temporal correctness becomes testable: if the question asks about fiscal year 2022, the retriever should surface the 2022 filing, not 2019.

Everything else from the proposal stayed the same — the KG²RAG backbone, the

validity-interval schema, the three-layer ablation. Only the corpus moved.
On the next slide, I'll show the research questions and where we stand today.

Research questions & experimental design

#	Question
RQ1	Where does KG ² RAG fail on temporal multi-hop QA?
RQ2	Does TempoRAG-KG lift F1 over KG ² RAG?
RQ3	How accurate is temporal extraction?
RQ4 ★	Can small-LM + temporal-KG $\approx$ large-LM alone?

RQ4 hypothesis: Temporal grounding closes the small-vs-large capability gap on temporal QA.

Test: 2 × 2 factorial · 7 models · 129 Qs

	No KG	+ KG
No Temporal	Vanilla	KG ² RAG
+ Temporal	TimeFilter	TempoRAG-KG

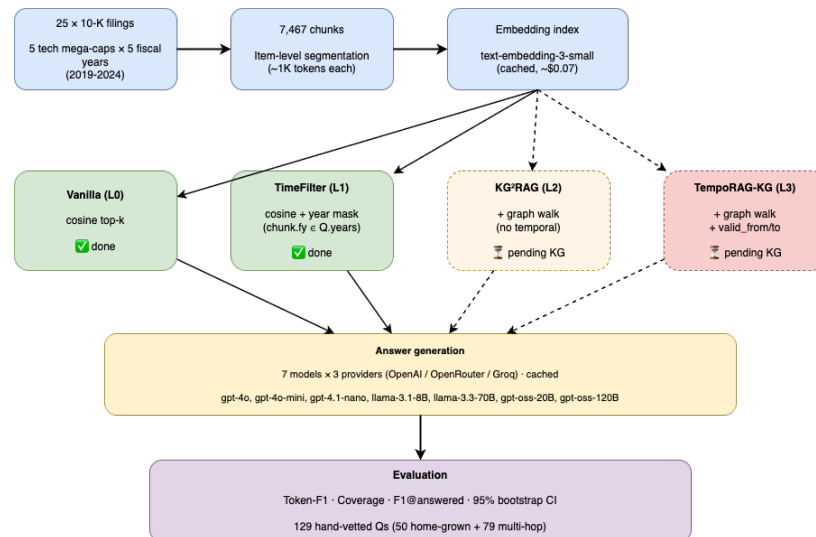
We have four research questions, RQ4 is new question after we have some change: can a small model paired with a temporal knowledge graph match what a large model does on its own? Our hypothesis is that temporal grounding closes the small-vs-large capability gap on temporal QA.

To test it, we run a 2 × 2 factorial design — with or without a knowledge graph on one axis, with or without a temporal filter on the other — across seven LLMs and 129 hand-vetted questions.

That gives us four conditions: Vanilla as the baseline, TimeFilter adds just the temporal filter, KG²RAG adds just the graph, and TempoRAG-KG is the full system combining both.

On the next slide, I'll show the pipeline that runs all four.

Pipeline



Here's the full pipeline, left to right. We start with 25 10-K filings from 2019 to 2024, segmented at the item level into about 7,500 chunks of roughly 1,000 tokens each, then embedded with OpenAI's text-embedding-3-small. The whole index cost about seven cents and it's fully cached.

From there, we fan out into the four retrieval conditions from the previous slide. Vanilla and TimeFilter — the two "no-KG" branches in green — are both done. The two "+KG" branches, KG²RAG and TempoRAG-KG, are waiting on the knowledge-graph build, which is currently running.

Every retrieved context then goes through the same answer-generation step — seven models across three providers, all cached — and ends in the same evaluation: token-F1, coverage, and F1 on answered rows, each with a 95% bootstrap confidence interval, on 129 hand-vetted questions.

On the next slide, I'll show the results for the two conditions we've completed.

Temporal filtering lifts F1 on all 7 models (+5.6% to +24.7%)

L0 Vanilla vs L1 TimeFilter · 129 Qs · 95% bootstrap CIs available

Model	Vanilla F1	+TimeFilter	Δ F1 %	Van cov	TF cov	Δ cov	F1@ans V→T
gpt-4.1-nano	0.229	0.258	+12.7%	63.6%	70.5%	+7.0pp	0.360 → 0.366
gpt-4o-mini	0.230	0.248	+7.8%	58.1%	62.8%	+4.7pp	0.396 → 0.395
gpt-4o	0.183	0.221	+20.9%	47.6%	56.6%	+9.0pp	0.385 → 0.391
llama-70b	0.145	0.179	+23.7%	43.4%	53.5%	+10.1pp	0.333 → 0.335
llama-8b	0.145	0.180	+24.7%	48.1%	57.4%	+9.3pp	0.301 → 0.314
gpt-oss-120b	0.131	0.138	+5.6%	35.7%	38.8%	+3.1pp	0.366 → 0.356
gpt-oss-20b	0.120	0.145	+21.0%	37.2%	42.6%	+5.4pp	0.320 → 0.336

Universal lift — 7/7 models improved (+5.6% to +24.7% F1).

Coverage drives F1 — Δ cov +3 to +10pp; F1@answered stays flat ≈ 0.35. Bottleneck is retrieval, not answer quality.

Small-model benefit — llama-8b +24.7%, gpt-oss-20b +21.0% vs gpt-4o-mini +7.8%. Early signal for RQ4 capability parity.

Here are our results so far — L0 Vanilla versus L1 TimeFilter, across 7 models and 129 questions.

The headline: adding a simple year-based temporal filter lifts F1 on every single model. The Δ-F1 column in green ranges from plus 5.6% at the low end to plus 24.7% at the high end. So the direction is unambiguous.

I want to call out three observations.

First — the lift is universal. Seven out of seven models improved. Not a single case where adding the filter hurt.

Second — coverage is what drives the F1 gain. The blue column shows coverage jumped 3 to 10 percentage points across the board, while F1 on answered rows stayed essentially flat around 0.35. That means the temporal filter isn't making answers more accurate — it's making more questions *answerable*. The bottleneck is retrieval, not answer quality. This is the first direct signal that a retrieval-level intervention is doing real work.

Third — small models benefit more than the mid-tier model. Llama-8B got plus 24.7%, GPT-OSS-20B got plus 21%, while GPT-4o-mini got only plus 7.8%. Small models were leaving the most on the table, and temporal filtering gives it back. That's the early directional signal for RQ4.

On the next slide: where we're *still* leaving performance on the table — which

motivates adding the knowledge graph.

Where TimeFilter hits its ceiling

gpt-4.1-nano slicing · motivates why we need KG next

Hop count	Vanilla F1	TimeFilter F1	Δ	n	Scope	Vanilla F1	TimeFilter F1	Δ	n
hop = 1	0.337	0.206	-0.131	10	inter_year	0.220	0.276	+0.055	53
hop = 2	0.234	0.281	+0.048	104	cross_company	0.264	0.315	+0.051	24
hop = 3	0.123	0.131	+0.007	15	intra	0.216	0.216	-0.001	45
					fiscal_vs_calendar	0.301	0.222	-0.079	5
					forward_looking	0.138	0.157	+0.019	2

TimeFilter shines on year-sensitive questions — inter_year +0.055, cross_company +0.051. Exactly where a year mask should help.

No lift on intra (same-year) questions — $\Delta \approx 0$. Year filter is a no-op when the question and all candidate chunks share one year.

hop = 3 barely moves — $\Delta = +0.007$ (n=15). Multi-hop reasoning needs entity cross-linking, which metadata alone cannot provide.

→ **This motivates KG²RAG and TempoRAG-KG** — graph walks add the cross-chunk relations TimeFilter is blind to.

This is the same gpt-4.1-nano data from the previous slide, but sliced by question type to show where the temporal filter helps and where it hits a ceiling. On the left, sliced by **hop count**: two-hop questions, which are 104 of the 129, got a solid plus 0.048. But three-hop questions barely moved — plus 0.007. The hop-1 dip looks alarming at minus 0.131, but that's only 10 questions, and those are easy cases where vanilla already scores 0.34.

On the right, sliced by **scope**: inter-year questions got plus 0.055, cross-company got plus 0.051 — exactly where a year mask *should* help. But intra-year questions, where the question and all candidate chunks share the same year, are completely flat. The filter has nothing to filter.

The takeaway: a metadata-level year filter works when *time* is the discriminating signal — but it's blind to multi-hop entity chains. Three-hop reasoning needs cross-chunk entity linking, and metadata alone cannot provide that.

That's exactly what KG²RAG and TempoRAG-KG add: graph walks across the cross-chunk relations TimeFilter is blind to. On the next slide, I'll show the state of that work.

What's next — KG extraction and the L2 / L3 lanes

KG extraction (running)

Stage	Detail
Corpus	7,467 chunks from 25 × 10-K filings
Extractor	gpt-4.1-nano, temporal-aware prompt
Schema	(subject, predicate, object, valid_from, valid_to)
Progress	Running (live + cached), mid-sweep
Budget	Well under \$20 cap

Remaining work

1. **Finish extraction** — complete full 7,467-chunk pass.
2. **Filter pass** — drop noisy triples (e.g. object="true", comparative artifacts).
3. **Build KG²RAG retriever** — seed chunk → graph walk → expanded context.
4. **Build TempoRAG-KG retriever** — KG²RAG + hard mask on valid_from / valid_to.
5. **Run L2 + L3 sweeps** — same 7 models × 129 Qs.
6. **Interaction analysis** — hop × scope × condition, test whether L3 > L1 + L2.

On the left: the knowledge-graph extraction that's currently running. We're pulling temporal triples — subject, predicate, object, plus valid_from and valid_to — out of all 7,467 chunks using gpt-4.1-nano with a temporal-aware prompt. Mid-sweep, well under the budget cap.

On the right: six remaining steps.

Once extraction finishes, a filter pass drops the noisy triples we've seen — things like object="true" and a few comparative-column artifacts.

Then we build the two missing retrievers: KG²RAG adds a graph walk from seed chunks, and TempoRAG-KG adds a hard mask on the validity intervals — the L2 and L3 cells of the 2×2 matrix.

With those in place, we run the same 7-model, 129-question sweep on both conditions, and finish with an interaction analysis — slicing by hop count and scope to test whether combining temporal filtering with the knowledge graph gives a super-additive gain. That's the core RQ2 test.

Takeaways

From this sprint · L0 + L1 baselines on 10-K temporal QA

- 1. TimeFilter baseline proven** — +5.6% to +24.7% F1 across 7 models; lift is universal, not model-specific.
- 2. Retrieval, not answer quality, is the bottleneck** — F1@answered stays flat around 0.35 while coverage moves +3 to +10pp. Any further lift must come from better retrieval.
- 3. Small models benefit more than large** — llama-8b +24.7% vs gpt-4o-mini +7.8%. Directional support for the RQ4 capability-parity story.
- 4. Multi-hop is next** — TimeFilter cannot lift hop = 3 (+0.007). KG²RAG and TempoRAG-KG will target exactly this gap.

To close, four takeaways from this sprint.

One. The temporal-filter baseline is now proven — plus 5.6 to plus 24.7 percent F1 across all 7 models. The lift is universal, not model-specific, which tells us the effect is structural — not a quirk of one architecture.

Two. The lift comes from coverage, not from answer quality. F1 on answered rows stays flat around 0.35 while coverage moves 3 to 10 percentage points. That means any further gain has to come from better retrieval — which is exactly why we're adding a knowledge graph next.

Three. Small models benefit more than the mid-tier one. Llama-8B plus 24.7% versus gpt-4o-mini's plus 7.8% — directional support for our RQ4 story, that the gap between small and large models narrows when retrieval is strong.

Four. Multi-hop is the clear next target. TimeFilter cannot move three-hop questions — only plus 0.007. That's precisely the gap KG²RAG and TempoRAG-KG are designed to close, and that's what we'll bring next time.

Thank you